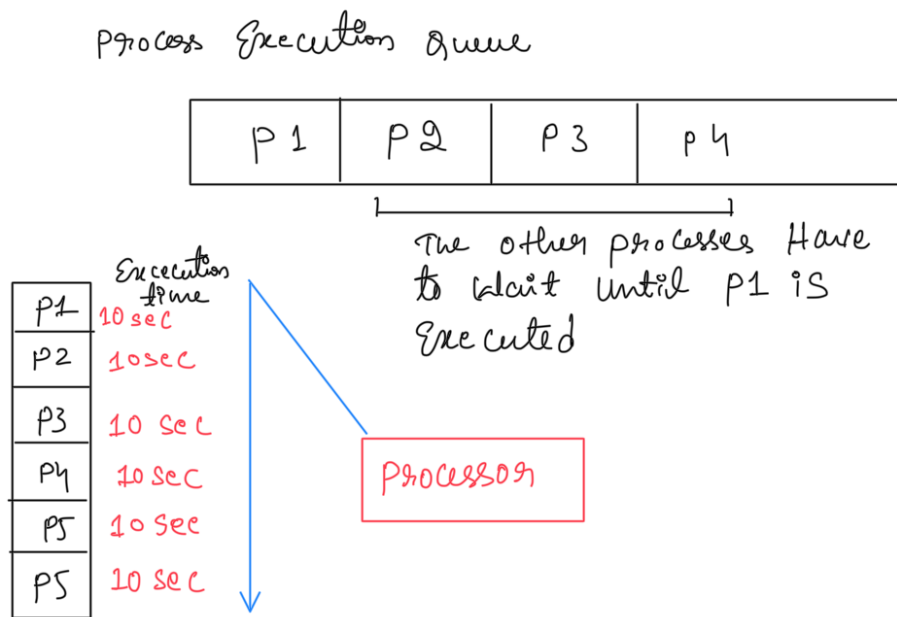


Process and Thread Scheduling in Operating System. (PART 2)

DAY 57
05/09/2025

Process Scheduling Algorithms

FCFS (First Come first Serve) → Processes are executed in the order they arrive.



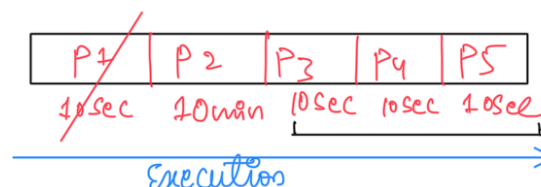
Similar to Queue data structure

No other processes will be executed until the First **Came** process is executed

Disadvantage of FCFS Algorithm

① if the execution time of all the processes are same then it doesn't make any difference.

But if any of the process have highest execution time among the same scheduled process then other processes have to wait long time until that process is executed



These three processes wait until 10 min Bcz P2 has to finish its execution

Even though P3 P4 P5 have less execution time they have to wait until P2 completes its execution. (That is called process starvation)

(2) Parallel processing is not possible → Process executes single process until it is completely executed.

◆ FCFS (First-Come, First-Served) Scheduling in OS

- **Definition:**
A CPU scheduling algorithm where the process that arrives first gets the CPU first.
- **Type:**
Non-preemptive (once a process starts, it runs until completion).
- **Queue Used:**
FIFO Queue (First-In, First-Out).

◆ How It Works

1. Processes enter the **ready queue** in the order they arrive.
2. CPU picks the **front process** in the queue.
3. After it finishes, the CPU picks the **next process** in arrival order.

◆ Advantages of FCFS

- ✓ Simple & easy to implement (FIFO queue).
- ✓ Fair: processes served in the order of arrival.
- ✓ Good for batch systems (long jobs acceptable).

◆ Disadvantages of FCFS

- ✗ **Poor average waiting time** if long process comes first (called **Convoy Effect**).
- Example: A long job delays all shorter jobs behind it.
- ✗ Not suitable for interactive systems (response time is high).
- ✗ Non-preemptive → cannot handle time-sensitive tasks well.

◆ Example

Suppose 3 processes arrive at time 0:

Process	Burst Time
P1	5 ms
P2	3 ms
P3	8 ms



Execution order: P1 → P2 → P3

• Completion Times:

- P1 = 5
- P2 = 8
- P3 = 16

• Waiting Times:

- P1 = 0
- P2 = 5
- P3 = 8

◆ What is Starvation?

Process is starving for CPU time

- **Starvation** = When a process waits indefinitely and never gets CPU time.
- Happens usually in **priority scheduling** (low priority processes may starve).
- In FCFS:
 - Strict starvation does **not** occur (because every process will eventually run).
 - But long waiting times (**convoy effect**) can feel *similar* to starvation for short processes.

◆ Real-World Computing Use Cases of FCFS

1. **Batch Processing in Early Operating Systems**
 - Old mainframes processed jobs in the order they arrived (batch jobs).
2. **Disk Scheduling (Simplest Form)**
 - If disk requests are served in the order they arrive → FCFS (though not optimal for performance).
3. **Network Packet Handling (Simple Routers / Switches)** 
 - Some basic network devices serve packets in the order they arrive.
4. **I/O Queue Management (Simple Devices)**
 - Devices like printers, plotters, and scanners often use FCFS to handle tasks.

Shortest Job next Algorithm / shortest Job First Algorithm

The process with the shortest Burst time is scheduled next

Advantages

Minimize Avg waiting time

disadvantage

May cause starvation
Hard to predict Burst time.

Shortest Job First Algorithm will also cause starvation

Process	Time
P1	10
P2	25
P3	4
P4	50
P5	3
P6	2
P7	10
P8	15

P4 will not be executed until the remaining smaller processes have completed their execution.

But the starvation will increase for longer processes.

Real-Life Analogies

1. Doctor's Clinic (Emergency Room)

- Patients with minor issues are treated first → quick turnover.
- But a patient with a serious condition (long treatment) may wait a long time (starvation).

2. Airport Check-in Counter

- If someone has only cabin baggage (short job), they finish quickly.
- Those with excess luggage (long job) may get delayed.

Real-World Use Cases

- Batch processing in OS → where job lengths are known.
- Embedded systems (like printers) → short tasks prioritized for fast response.
- Background job scheduling (compilation, batch jobs in mainframes).

Advantages

- Minimum Average Waiting Time (provably optimal).
- Good for batch systems where jobs are known in advance.

Disadvantages

- Starvation: Long processes may never get CPU if short jobs keep coming.
- Difficult to predict burst time (OS must estimate using past CPU usage → called exponential averaging).
- Not suitable for interactive systems (delays for large jobs).

Priority scheduling

- Each process is assigned priority

Pros → Important tasks get more CPU time

Cons → Starvation of low priority processes.

Based on the priority operating system will decide who will execute first I mean which process will be executed first.

Example

Let's say we have 4 processes:

Process	Burst Time	Priority (1 = highest)
P1	10	3
P2	1	1
P3	2	4
P4	1	2

◆ Non-Preemptive Execution Order

- At time 0 → among all, P2 (priority=1) runs first.
- Next → P4 (priority=2).
- Next → P1 (priority=3).
- Last → P3 (priority=4).

👉 Order: P2 → P4 → P1 → P3

📌 Types

1. Preemptive Priority Scheduling

- If a new process arrives with a higher priority than the currently running process, the CPU is preempted and given to the new process.

2. Non-Preemptive Priority Scheduling

- Once a process starts, it will continue till completion, even if a higher priority process arrives.

◆ Preemptive Execution Order

- At time 0 → P1 starts (burst=10, priority=3).
- At time 1 → P2 arrives (priority=1, higher than P1) → preempts P1.
- After P2 finishes, P4 (priority=2) runs.
- Then P1 resumes, followed by P3.

👉 Order: P1 → P2 → P4 → P1 → P3

⚡ Solution to Starvation

- **Aging:** Increase the priority of a process the longer it waits in the queue. This ensures that every process eventually gets executed.

🏠 Real-Life Analogies

1. Hospital Emergency Room 🏥
 - Critical patients are treated first (high priority), while minor issues wait.
2. Air Traffic Control ✈️
 - Emergency landings get the highest priority, while normal flights wait.

🌐 Real-World Use Cases

- **Operating Systems:** Interrupt handling (I/O devices get higher priority).
- **Real-Time Systems:** Safety-critical tasks like airbags, pacemakers, or sensors.
- **Network Scheduling:** High-priority packets (like VoIP, video streaming) processed first over normal data packets.

✅ Advantages

- High-priority tasks get executed quickly.
- Useful in real-time systems where some processes are more critical (e.g., handling interrupts, emergency signals).

❌ Disadvantages

- **Starvation (Indefinite Blocking):**
Low-priority processes may never get executed if high-priority processes keep arriving.
 - Example: In an airport, VIP flights (high priority) always get clearance first → normal flights (low priority) may wait endlessly.
- Requires a fair priority assignment policy; otherwise, unfair execution.

Every Process Have
Priority given

if keep on High priority
tasks Executed By processor
Suppose any new process
Came Have Highest priority
Then it will Be Executed

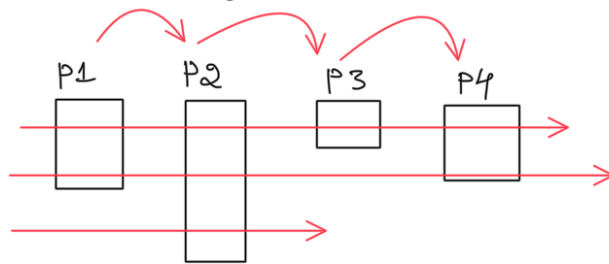
If Every new process Entered Have Highest
priority. then the old process will keep wait
state.

The Process with low priority will
keep on kept in the wait

lowest priority tasks goes
for starvation.

Round Robin

Each process get a fixed time slice



process's here
Executed in timeslice

Best way to understand this algorithm is How panipuri
guy serves Panipuri to Each Customer in particular time
slice

Same thing
Happens in a
Round Robin
Algorithm



This panipuri
Seller serves
the Panipuri
One By one
untill they
Done with Eating
Panipuri

The processor executes the process for a particular time
slice and moves to next process this is repeated until
The the process is completely executed.

disadvantage of Round Robin is Over Head of Context switches

3. Context Switching Overhead

Overhead means: work done by CPU that does NOT contribute to actual process execution.

During a context switch, CPU is busy with:

1. Saving registers, PC, SP, etc. of the old process.
2. Flushing pipelines, caches, and sometimes TLB.
3. Updating OS data structures (ready queue, PCB).
4. Restoring registers, PC, SP, etc. of the new process.

But none of this is user computation.

So context switching = pure overhead.

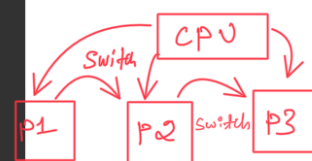
To save or restore actions

1. What is Context Switching?

A process (or thread) runs on the CPU with its own state:

- Program Counter (PC) → where execution will resume.
- CPU Registers → current working variables.
- Stack Pointer (SP) → location in memory stack.
- Memory mappings (Page table, TLB).

Processes need some
time to switch
from one process to
another



◆ 6. Analogy

Think of a teacher switching between students in a classroom:

- She must **note down what one student was doing**.
- Recall what the next student was doing.
- If she switches too frequently, she wastes more time **tracking progress** than actually **teaching**.

That wasted effort = **context switching overhead**.

✓ Final Takeaway

- Context switching is necessary for multitasking.
- But it is **overhead** (wasted CPU cycles).
- Minimized by:
 - Larger time quantum (but not too large).
 - Efficient scheduling algorithms.
 - Hardware support (fast TLB, register sets).

◆ 4. Cost of Context Switching

(a) Time Cost

- Depends on hardware and OS design.
- Typical range: **a few microseconds to a few milliseconds**.

Example:

If context switch = 1 ms and you do it 1000 times per second, then:

- CPU wastes 1000 ms = 1 second every second!
- Effectively, 0% CPU is left for real work.

(b) CPU Utilization Loss

Formula (for Round Robin scheduling):

$$\text{CPU Utilization} = \frac{\text{Time Quantum}}{\text{Time Quantum} + \text{Context Switch Time}}$$

Example:

- Time quantum = 10 ms
- Context switch = 1 ms

$$\text{CPU Utilization} = \frac{10}{10 + 1} = 90.9\%$$

If time quantum = 2 ms:

$$\text{CPU Utilization} = \frac{2}{2 + 1} = 66.6\%$$

👉 Too small quantum → huge overhead.

◆ 3. Disadvantages of RR

1. Context Switching Overhead

- Frequent switches if time quantum is small → CPU wastes time.

2. Time Quantum Sensitivity

- Too small → more overhead, poor CPU efficiency.
- Too large → degenerates into FCFS, loses interactivity benefits.

3. Poor Average Turnaround Time

- Compared to SJF, RR may have higher turnaround for short processes.

4. Not Optimal for CPU-Bound Processes

- CPU-heavy tasks may wait longer than needed.

◆ 2. Advantages of RR

1. Fairness

- Each process gets an equal opportunity to run.
- No starvation for processes.

2. Good for Interactive Systems

- Responsive because processes are switched frequently.
- Users feel the system is fast (good for GUI, servers).

3. Simple to Implement

- Easy queue-based logic: just maintain a ready queue and rotate.

4. Predictable Time Sharing

- Maximum wait time $\leq (n-1) \times \text{time quantum}$, where n = number of processes.

Multilevel Queue scheduling

Processes are divided into queues Example: Batch
enter Active

PROS

- Special Handling for different types of processes.

CONS

- Rigid Queue Boundaries. (not easy to jump from one queue to another)

◆ 1. Overview of Multilevel Queue Scheduling

Multilevel Queue Scheduling (MQS) is an extension of CPU scheduling where:

- Processes are grouped into multiple queues based on priority, type, or characteristics.
- Each queue can have its own scheduling algorithm (e.g., FCFS, RR).
- Queues are prioritized, and higher-priority queues are executed before lower-priority queues

Use case example:

- System processes → highest priority → FCFS
- Interactive processes → medium priority → Round Robin
- Batch processes → low priority → FCFS

◆ 2. Characteristics

1. Multiple Queues

- Each queue contains processes of a similar type.
- Example: foreground (interactive) vs background (batch).

2. Queue Scheduling

- Scheduling happens within each queue using an algorithm suitable for that queue

3. Queue Priorities

- Higher-priority queues are served first.
- Lower-priority queues wait until higher-priority queues are empty.

◆ 3. Types of Multilevel Queue Scheduling

1. Fixed Priority Scheduling

- Each queue has a fixed priority.
- Higher-priority queues always preempt lower-priority queues.
- Risk: **Starvation** for low-priority queues.

2. Time Slice (Feedback)

- Some systems implement **multilevel feedback queues**:
 - Processes can move between queues based on their behavior.
 - Helps prevent starvation and favors short interactive jobs.

◆ 4. Advantages

1. Separation of Process Types

- Easy to manage processes with different characteristics.

2. Flexibility

- Each queue can use a different scheduling algorithm suitable for that type

3. Better Response for Interactive Processes

- Interactive processes can be in a high-priority queue.

◆ 5. Disadvantages

1. Starvation

- Low-priority queues may never get CPU if higher-priority queues are always busy.

2. Complex Implementation

- Need to maintain multiple queues and scheduling logic.

3. Queue Assignment

- Assigning a process to the correct queue can be tricky.

processes can't switch from one queue to another queue until all the processes in the queue are executed completely

◆ 6. Real-life Use Case

• Operating Systems (OS):

- Windows and UNIX often use multilevel queues for separating system, interactive, and batch processes

• Web Servers:

- Requests can be categorized into interactive (high priority) and batch (low priority) tasks.

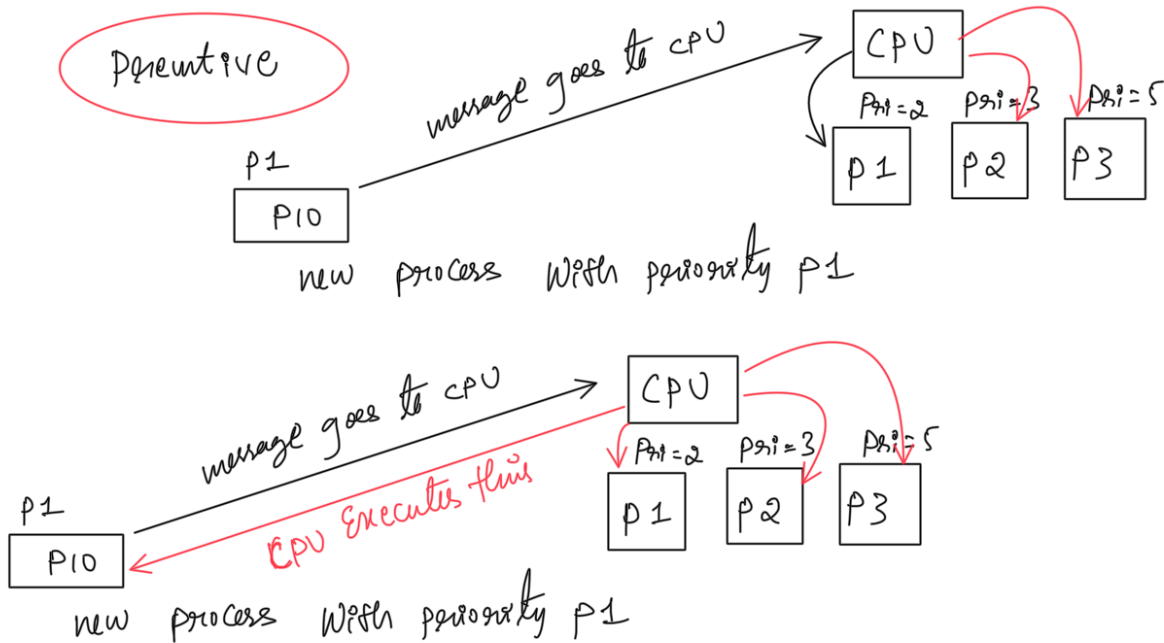
Multilevel Feedback Queue

Solves the Queue Switching problem.

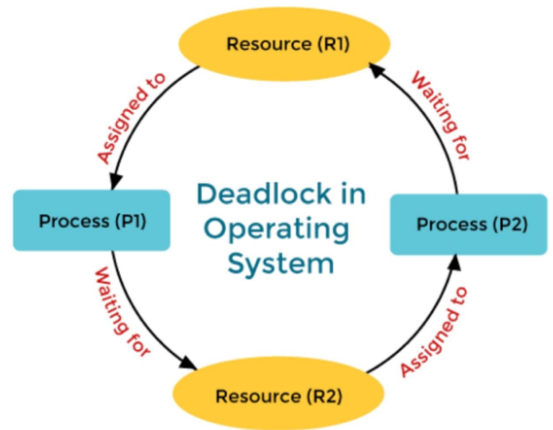
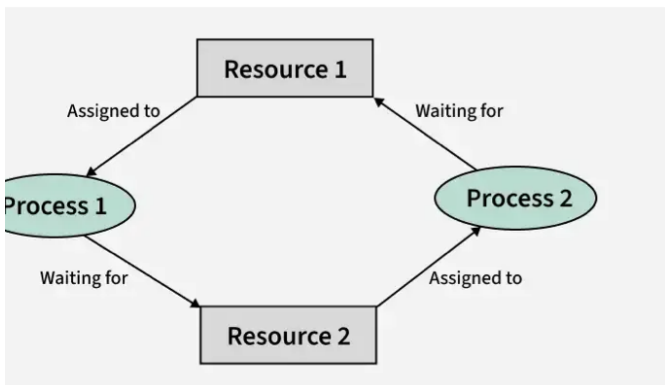
Processes can move between Queues Based on Behaviour

Pros → More flexible than multilevel queue

Cons → Complex to Implement.



Dead Lock



All processes are waiting and nothing is gonna execute

So critical section is created

like at particular time of process execution no other process can make change on a file in which already a process is making some changes.

If process is aged and its waiting for CPU for long time then the CPU will execute that process.

The Blue Screen of Death (BSOD) is a critical error screen displayed by Windows when the operating system encounters a system-level fault it cannot recover from. Let's break it down:

1. What is BSOD?

- BSOD is an error screen with a blue background that appears when Windows encounters a **fatal system error** (also called a *stop error*).
- It **halts all operations** to prevent damage to the system or data.
- Shows an **error code** and sometimes the name of the driver or module that caused the issue.

3. What Happens During BSOD

1. Windows detects an unrecoverable error.
2. The system **stops all operations** to avoid data corruption.
3. Memory dump is created (can be used for debugging).
4. Blue screen appears with error information (e.g., `0x0000007B`).
5. System usually **reboots automatically** after displaying the error.

4. Key Terms in BSOD

- **Stop Code:** Identifies the type of error (e.g., `PAGE_FAULT_IN_NONPAGED_AREA`).
- **Bug Check:** Internal code Windows uses to classify the error.
- **Memory Dump:** Snapshot of memory for debugging the cause.



2. Common Causes

1. Hardware Issues

- Faulty RAM, hard drives, or overheating CPU/GPU.

2. Driver Problems

- Outdated, corrupt, or incompatible device drivers.

3. Software Conflicts

- Improperly installed software, especially low-level system utilities

4. Malware

- Some malware can cause critical system faults.

5. Corrupt System Files

- Windows system files (like `ntoskrnl.exe`) can get damaged.

5. How to Fix or Prevent BSOD

1. Update Drivers and Windows

2. Check Hardware: Test RAM, hard drives, and overheating components.

3. Scan for Malware

4. System Restore / Recovery

5. Check for Software Conflicts

6. Analyze Memory Dump (using tools like WinDbg)

Real-Life Example:

- You install a new graphics driver, restart, and Windows suddenly shows BSOD with a stop code pointing to `nvlddmkm.sys` (NVIDIA driver). This indicates a **driver conflict** or corruption.

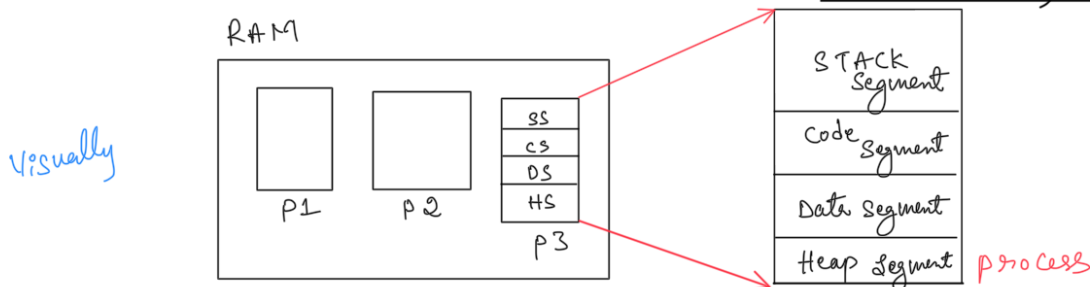
Process in RAM and kernel structures

• RAM allocation

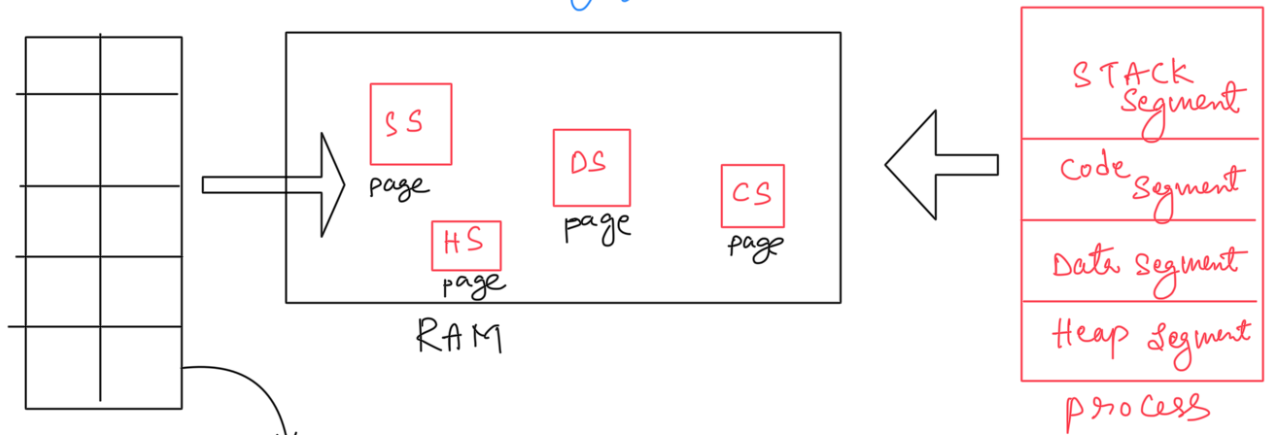
- Code Segment
- Data Segment
- Stack
- Heap

• Process Table

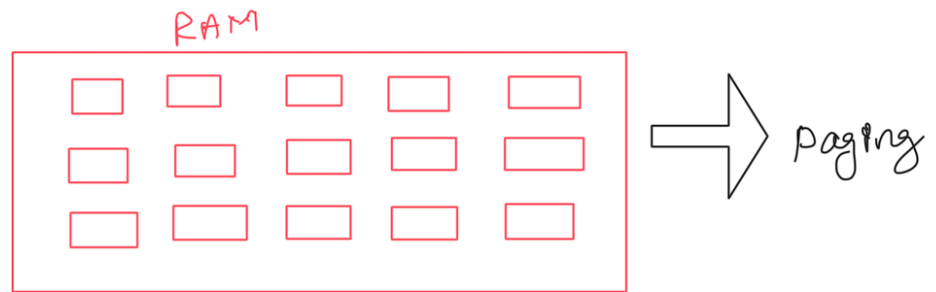
- Maintained in kernel space
- Contains PID, state, registers, priority, memory pointers



conceptually its in paging form



Every process made into smaller pages



Tomorrow I will learn about threads in depth.